前端学习路线 | 26 年最新零基础到精通一条龙（万人收藏⭐）

> *编程导航学习网站：学编程、做项目、拿 Offer！*
>
> *企业高频面试题库：开始刷题，面试遇原题！*
>
> *精选简历模板大全：1 分钟搞定简历！*
>
> *AI 资源导航网站：获取最新 AI 黑科技！*
>
> *1 对 1 模拟面试：随时随地提升面试能力*

前端开发求职高频面试题：开始刷题

⭐ 建议先观看鱼皮的视频导读：2025 年最新保姆级前端学习路线

开篇介绍

前端开发，简单来说就是 **开发用户能看到、能交互的网页界面**。从静态的展示页面，到动态的 Web 应用，再到如今的复杂单页应用（SPA）、移动端 H5、小程序、桌面应用，前端的应用领域越来越广，技术栈也越来越丰富。

为什么要学前端呢？

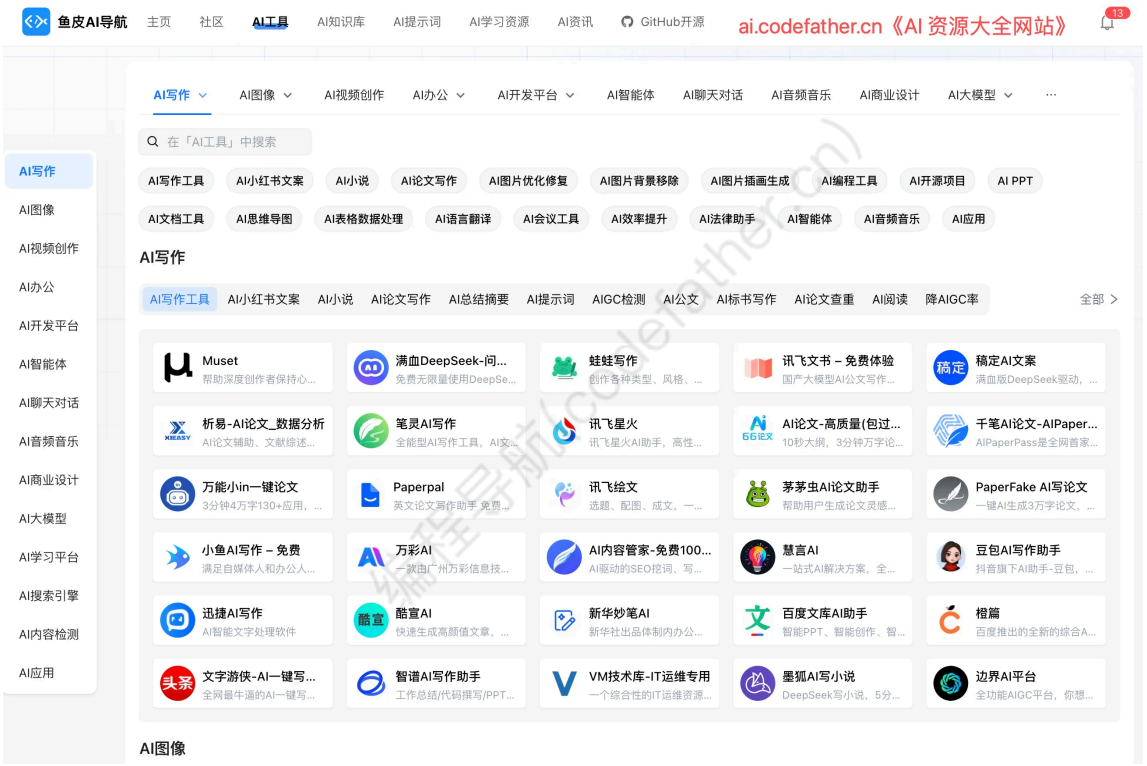
首先，前端是编程领域中最容易入门的方向之一，因为能立刻看到效果，写几行代码就能在浏览器里呈现出漂亮的页面，非常有成就感！其次，前端的岗位需求量大，无论是互联网公司还是传统企业，只要有网站、有 App，就需要前端工程师。第三，前端的技术生态非常活跃，新技术层出不穷，能让你保持学习的热情。

学好前端，你能做什么呢？

可以开发企业官网、电商平台、管理系统、移动端 H5 页面、微信小程序、桌面应用，甚至是游戏界面、数据可视化大屏等等，应用场景非常广泛。

特别是在 AI 时代，前端开发也有了新的机会！AI 应用需要友好的用户界面，需要前端工程师来开发 AI 对话界面、AI 内容生成页面等。而且现在有很多 AI 辅助编程工具，能大大提升前端开发效率（Vibe Coding），让前端专注于做更有挑战的工作。掌握前端 + AI，将会拥有更强的竞争力。想了解更多 AI 工具和资源，可以访问鱼皮的 AI 资源大全。

编程导航(codefather.cn)



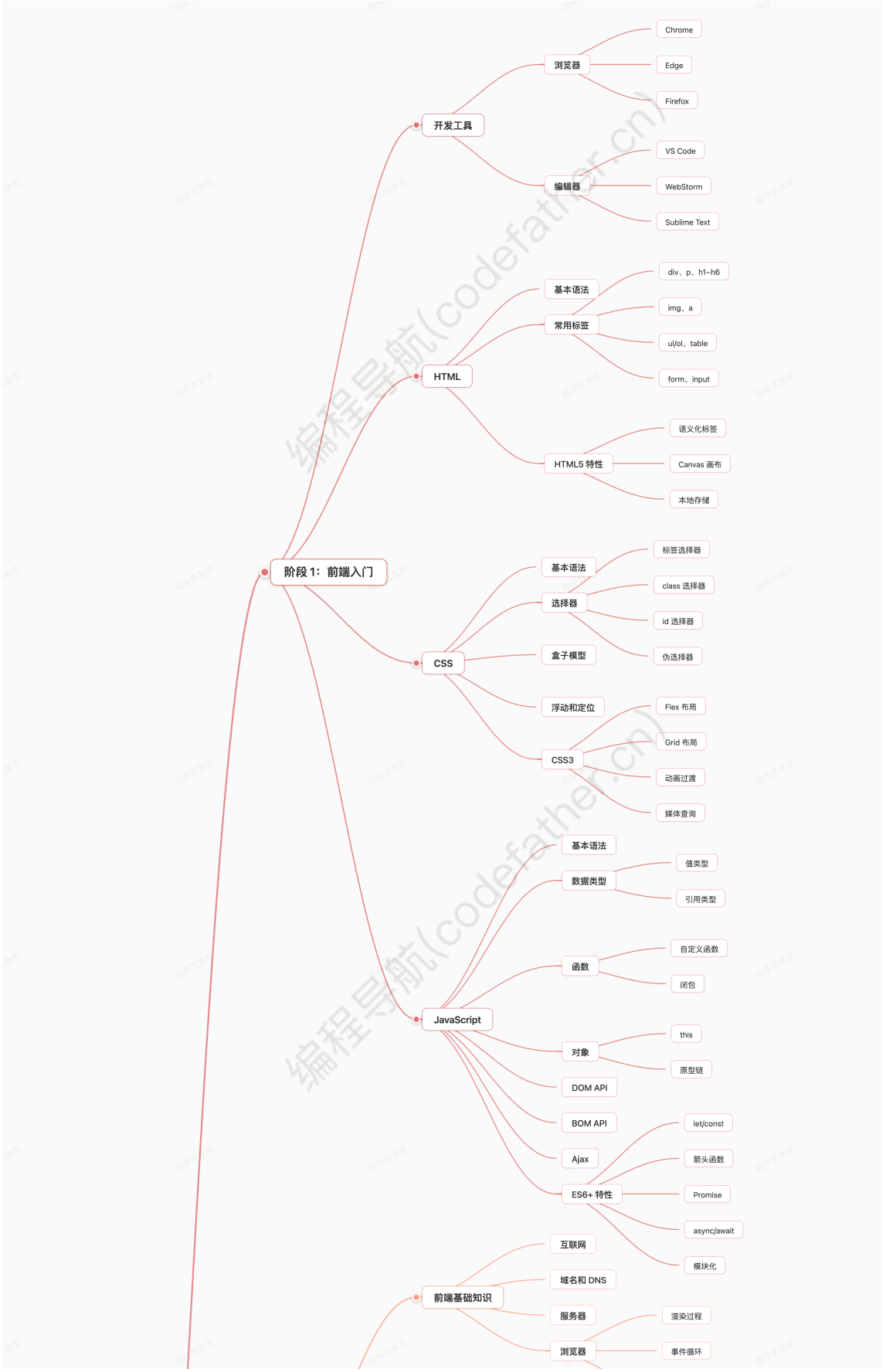
就业方向

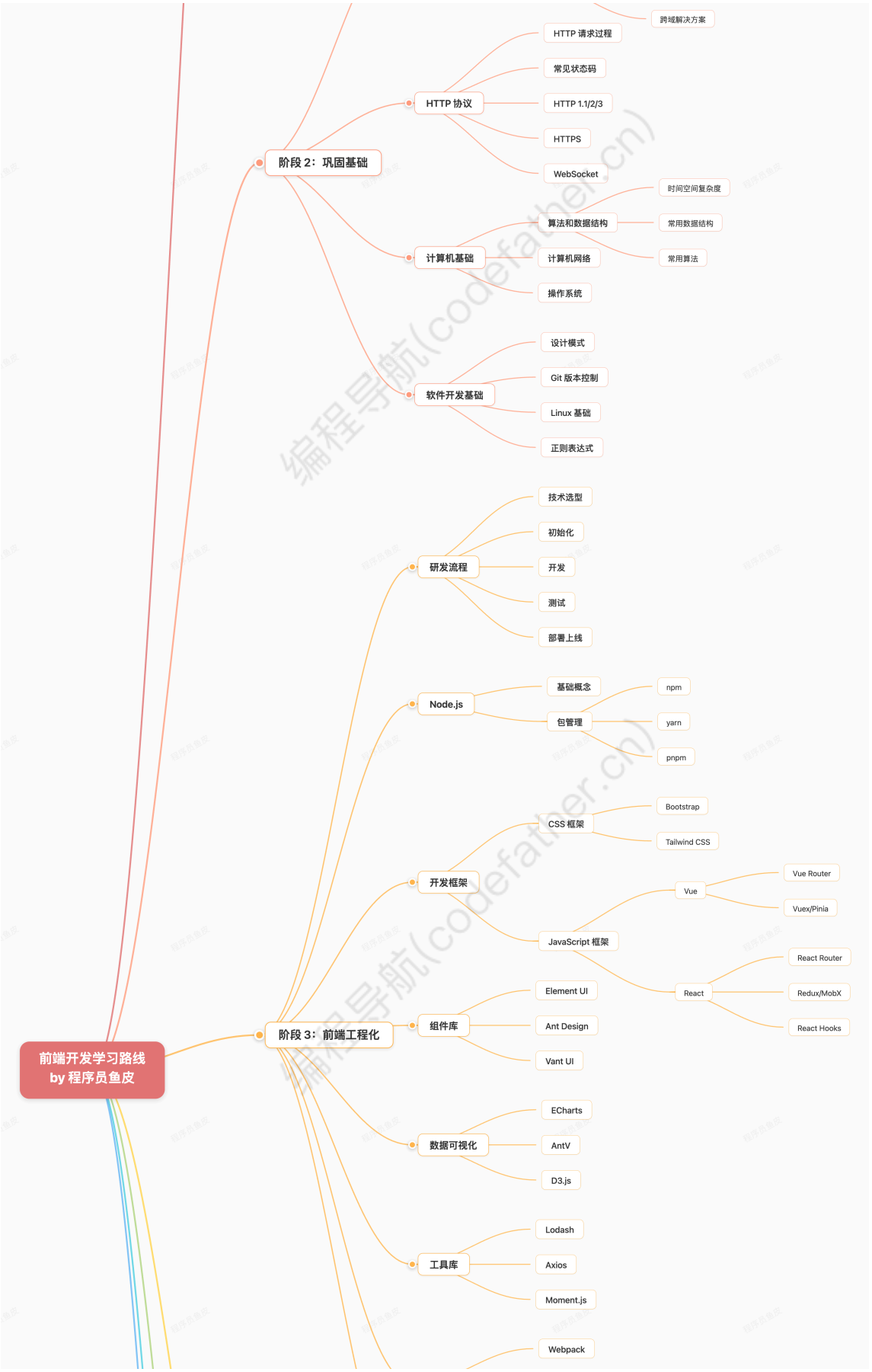
学习前端开发后，常见的就业方向包括：

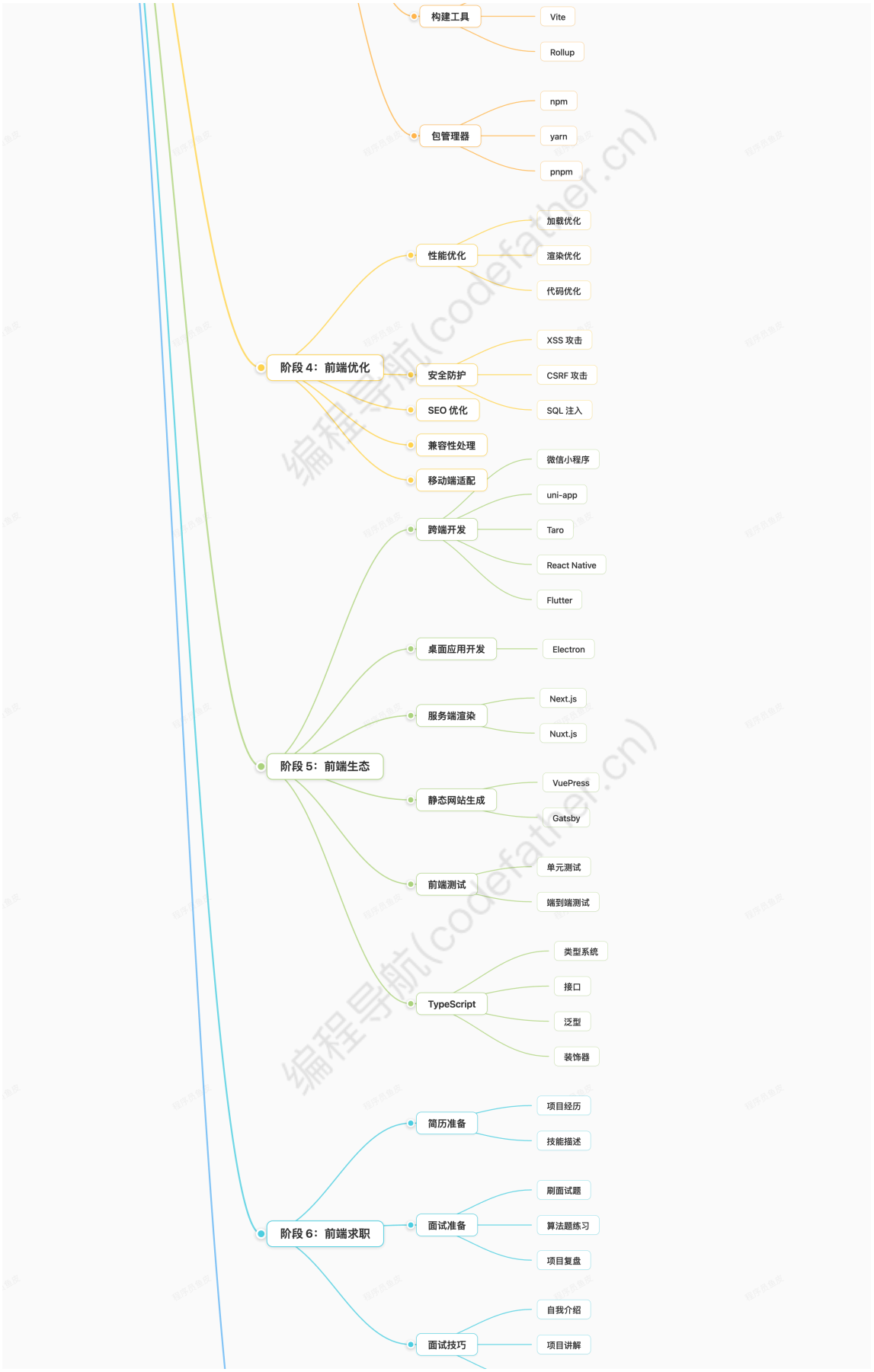
1. Web 前端开发工程师：负责网站前端页面开发，这是前端最主流的就业方向。工作内容包括还原设计稿、开发交互功能、前后端联调、性能优化等。需要熟练掌握 HTML、CSS、JavaScript 以及至少一个主流框架（Vue 或 React）。
2. 移动端 H5 开发工程师：专注于开发移动端 H5 页面，需要熟悉移动端适配、触摸事件、移动端性能优化等。一般需要掌握 Vue 或 React，以及移动端 UI 框架（如 Vant）。
3. 全栈开发工程师：既能做前端也能做后端，能够独立完成整个项目的开发。需要在前端的基础上，额外学习后端技术（Node.js、Java、Go 等）和数据库。关于 Node.js 的详细学习，可以查看 Node.js 学习路线。
4. 小程序开发工程师：开发微信小程序、支付宝小程序等。需要熟悉小程序框架（原生或 uni-app、Taro 等跨端框架）。
5. 前端架构师：在有丰富的前端开发经验后，可以往架构师方向发展，负责前端技术选型、工程化建设、性能优化、团队规范制定等。
6. 可视化工程师：专注于数据可视化、3D 可视化等方向，需要熟悉 ECharts、D3.js、Three.js 等可视化库。
7. 低代码平台开发工程师：开发低代码平台，让非技术人员也能通过拖拽组件来创建应用。需要深入理解前端工程化、组件化、编译原理等。

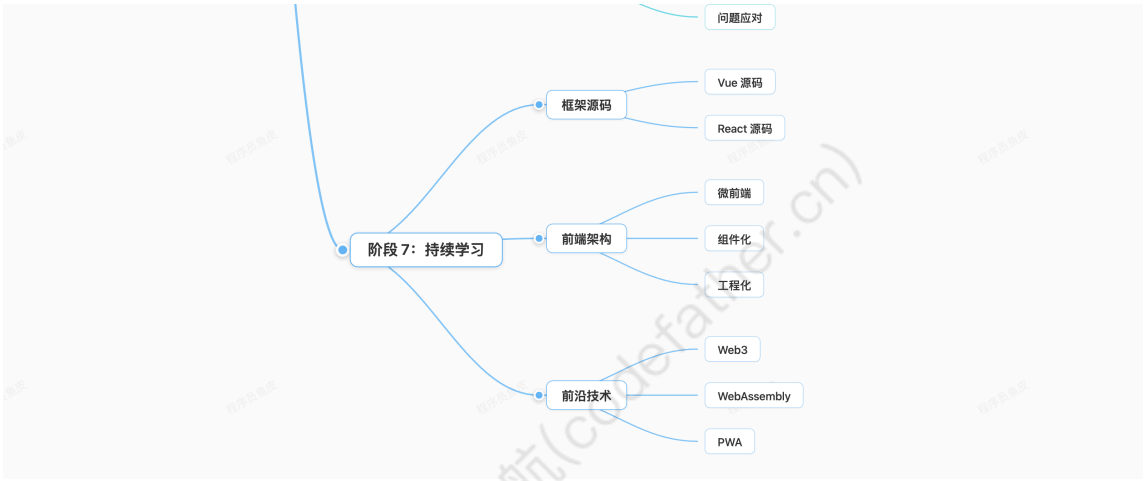
学习路线图

下面是完整的前端学习路线图，帮助大家理清学习脉络：


阶段 1：前端入门
开发工具
浏览器
Chrome
Edge
Firefox
编辑器
VS Code
WebStorm
Sublime Text
HTML
基本语法
常用标签
div、p、h1~h6
img、a
ul/ol、table
form、input
HTML5 特性
语义化标签
Canvas 画布
本地存储
CSS
基本语法
选择器
标签选择器
class 选择器
id 选择器
伪选择器
盒子模型
浮动和定位
CSS3
Flex 布局
Grid 布局
动画过渡
媒体查询
JavaScript
基本语法
数据类型
值类型
引用类型
函数
自定义函数
闭包
对象
this
原型链
DOM API
BOM API
Ajax
ES6+ 特性
let/const
箭头函数
Promise
async/await
模块化
前端基础知识
互联网
域名和 DNS
服务器
浏览器
渲染过程
事件循环


跨域解决方案
HTTP 请求过程
常见状态码
HTTP 协议
HTTP 1.1/2/3
HTTPS
WebSocket
阶段 2：巩固基础
时间空间复杂度
算法和数据结构
常用数据结构
计算机基础
计算机网络
常用算法
操作系统
设计模式
Git 版本控制
软件开发基础
Linux 基础
正则表达式
技术选型
初始化
研发流程
开发
测试
部署上线
基础概念
npm
Node.js
包管理
yarn
pnpm
Bootstrap
CSS 框架
Tailwind CSS
开发框架
Vue
Vue Router
Vuex/Pinia
JavaScript 框架
React
React Router
Redux/MobX
React Hooks
前端开发学习路线
by 程序员鱼皮
阶段 3：前端工程化
Element UI
组件库
Ant Design
Vant UI
ECharts
数据可视化
AntV
D3.js
Lodash
工具库
Axios
Moment.js
Webpack


构建工具
Vite
Rollup
包管理器
npm
yarn
pnpm
阶段 4：前端优化
性能优化
加载优化
渲染优化
代码优化
安全防护
XSS 攻击
CSRF 攻击
SQL 注入
SEO 优化
兼容性处理
移动端适配
跨端开发
微信小程序
uni-app
Taro
React Native
Flutter
阶段 5：前端生态
桌面应用开发
Electron
服务端渲染
Next.js
Nuxt.js
静态网站生成
VuePress
Gatsby
前端测试
单元测试
端到端测试
TypeScript
类型系统
接口
泛型
装饰器
阶段 6：前端求职
简历准备
项目经历
技能描述
面试准备
刷面试题
算法题练习
项目复盘
面试技巧
自我介绍
项目讲解



整体学习建议

1）相信自己有能力：前端是最容易入门的编程方向，不要被各种新技术吓到。相信自己有能力学会，那么你就真的会学会！

2）前端学习一定要以培养兴趣为主，不要一上来就啃理论。建议从实战开始，边学边做，看到自己写的代码在浏览器里呈现出效果，会很有成就感。

3）善用 AI 工具：AI 时代学前端太幸福了！可以让 AI 帮你生成 HTML/CSS 代码、解释 JavaScript 概念、Debug 报错、甚至生成整个页面 / 项目。特别推荐跟着鱼皮的 AI 程序员技术练兵场项目 学习如何用 AI 辅助前端开发。



4）多敲代码，多做项目：前端学习不能靠死记硬背，一定要多敲代码！建议跟着视频把每个例子都敲一遍，然后自己做一些小项目。忘了就查文档，不要强行记忆。

5）循序渐进，不要贪多：前端的知识点非常多，不要想着一下子全学会。按照路线一步步来，先打好基础、再学框架、再做项目、最后拓展视野。

阶段 1：前端入门（20-45 天，仅供参考）

这是前端开发的入门阶段，主要学习前端三件套：HTML、CSS、JavaScript。HTML 是网页的骨架，CSS 是网页的样式，JavaScript 是网页的灵魂。掌握这三个技术，你就能开发出基本的网页了。

学习目标

了解和实践前端三件套（HTML、CSS、JavaScript）的基础语法，并能使用开发工具独立开发一个留言板网站。

知识点

开发工具【必学】：

工欲善其事，必先利其器。

- 浏览器
  - Chrome【必学】
  - Edge【建议学】
  - Firefox【建议学】
  - Safari
- 编辑器
  - VS Code【必学】
  - WebStorm【建议学】
  - Sublime Text【建议学】
- 文档笔记
  - Markdown 语法【建议学】
  - Typora【建议学】
  - 在线笔记（语雀、腾讯文档、石墨文档等）

HTML【必学】：

HTML 用于定义网页的结构，是前端的基础。

- 基本语法【必学】
- 常用标签【必学】
  - 分区 div
  - 标题 h1 ~ h6
  - 段落 p
  - 图像 img
  - 列表 ul / ol
  - 超链接 a
  - 表单 form
  - 表格 table
  - 框架 iframe
- 属性【必学】
- HTML5 特性【建议学】
  - 语义化标签
  - 浏览器支持
  - 多媒体标签
  - Canvas 画布
  - 本地存储
    - localStorage
    - sessionStorage
  - Web Workers【可不学】

编程导航(codefather.cn)

    - 应用缓存【可不学】

CSS【必学】：

CSS（层叠样式表）用于设计网页的风格和布局，让网页变得美观。

- 基本语法【必学】
- 引入方式【必学】
    - 行内样式
    - 内部样式表
    - 外部样式表
- 选择器【必学】
    - 通用选择器
    - 标签选择器
    - id 选择器
    - class 选择器
    - 属性选择器
    - 派生选择器（后代、子元素、相邻兄弟）
    - 组合选择器
    - 伪选择器
    - 选择器优先级
- 常用属性【必学】
    - 单位（px、em、rem、vw、vh）
    - 背景、文本、字体
    - 列表、表格
- 文档流【必学】
    - 标准流
    - 浮动流
    - 定位流
- 盒子模型【必学】
    - content、padding、border、margin
- 浮动【必学】
    - 设置浮动 float
    - 清除浮动 clear
- 定位【必学】
    - static、absolute、fixed、relative、sticky
- 层叠规则【必学】
- BFC 和 IFC 机制【建议学，面试重点】
- CSS3【建议学】
    - 响应式布局【必学】
        - 媒体查询
        - Flex 布局
        - Grid 布局
    - 动画、过渡、渐变
    - 圆角、阴影、背景
    - 2D / 3D 转换【可不学】

JavaScript【必学】：

JavaScript 是前端的灵魂，具有函数优先的轻量级编程语言，用于实现网页的交互功能。

编程导航(codefather.cn)
编程导航(codefather.cn)

- 基本语法【必学】
- 数据类型【必学】
  - 值类型
    - 字符串 String
    - 数组 Number
    - 布尔 Boolean
    - 空值 Null
    - 未定义 Undefined
    - Symbol
  - 引用类型
    - 对象 Object
    - 数组 Array
    - 函数 Function
- 数据类型转换【建议学】
- 函数【建议学】
  - 概念【必学】
  - 自定义函数【必学】
  - 调用方式
  - 全局调用【必学】
  - 构造函数调用
  - 函数方法调用【必学】
  - apply、call【建议学】
  - 闭包【建议学，面试重点】
- 对象【建议学】
  - 概念
  - this
  - 原型链和继承
  - 常用对象
    - 数字 Number
    - 字符串 String 对象
    - 日期 Date 对象
    - 数组 Array
    - 布尔 Boolean
    - 算数 Math
  - 自定义对象
- 作用域和作用域链【建议学，面试重点】
- BOM API【建议学】
- DOM API【必学】
- JSON【必学】
- Ajax【必学】
- JavaScript 执行机制【建议学】

ES6+ 特性【建议学】：

ES6+ 是 JavaScript 的新语法特性，现代前端开发必须掌握。

- let 和 const【必学】
- 变量解构赋值【必学】
- 箭头函数【必学】
- 模板字符串【必学】

编程导航(codefather.cn)
编程导航(codefather.cn)

- 对象扩展和新增方法【建议学】
- Symbol【可不学】
- Set 和 Map 数据结构【建议学】
- Promise 和 async/await 异步编程【必学，面试重点】
- Generator 函数【可不学】
- 模块化（import/export）【必学】

学习建议

1）重点是 JavaScript：HTML 和 CSS 相对简单，几天就能学会。JavaScript 是前端的灵魂，一定要多花时间学，多写代码练习。

2）善用浏览器开发者工具：学会使用 Chrome 开发者工具调试代码，能大大提升学习和开发效率。

学习资源

- Pink 老师前端入门教程：零基础必看，H5 + CSS3 + 移动端，讲得非常详细
- 2025 最新 Web 前端零基础入门：HTML5 + CSS + JS 全套，140 集
- MDN Web 开发文档：前端官方文档，必读
- freeCodeCamp 在线编程：在线实战学习，强烈推荐
- 阿里云前端实战学习：在线动手实验
- MDN HTML 教程：HTML 官方教程
- MDN CSS 教程：CSS 官方教程
- MDN JavaScript 教程：JavaScript 官方教程
- ES6 入门教程：阮一峰老师的 ES6 教程

阶段 2：巩固基础（10-30 天，仅供参考）

在掌握了前端三件套的基本语法后，这个阶段要深入学习 JavaScript 的高级特性，并了解浏览器、HTTP、计算机基础等知识，为实战开发打下坚实的基础。

学习目标

熟悉前端三件套语法（尤其是 JavaScript），并了解互联网、域名、浏览器、服务器等基础知识，扎实前端程序员的基本功，为进入实战开发做准备。

知识点

前端基础知识【建议学】：

- 互联网【建议学】
- 域名【必学】
- DNS【必学】
- 服务器【必学】
- 浏览器【建议学】
  - 浏览器 DOM 事件流 / 事件委托
  - 浏览器加载顺序【必学】
  - 浏览器渲染过程【必学，面试重点】
  - 浏览器事件循环【建议学】
  - 浏览器同源策略【建议学】
  - 跨域解决方案【必学，面试重点】
  - 浏览器缓存【建议学】
  - 控制台调试技巧【必学】
- HTTP 协议【建议学】

编程导航(codefather.cn)

    - HTTP 请求过程【建议学，面试重点】
    - 常见 HTTP 协议
        - HTTP 1.0
        - HTTP 1.1【必学】
        - HTTP 2【建议学】
        - HTTP 3【可不学】
    - HTTP 请求类别【必学】
    - 常见状态码【必学】
        - 1xx 信息
        - 2xx 成功
        - 3xx 重定向
        - 4xx 客户端错误
        - 5xx 服务器错误
    - WebSocket【建议学】
    - Cookie【必学】
    - Session【建议学】
    - HTTPS【必学】

计算机基础【建议学】：

- 算法和数据结构
    - 时间/空间复杂度分析【必学】
    - 数据结构：数组、字符串、队列、栈、链表、集合、哈希表、二叉树
    - 算法：排序、双指针、查找、分治、动态规划、递归、回溯、贪心、位运算、DFS、BFS、图
- 计算机网络
    - HTTP 协议
    - 网络模型
    - UDP / TCP 协议
- 操作系统【可不学】
    - 进程、线程
    - 进程 / 线程间通讯方式
    - 进程调度算法
    - 进程 / 线程状态
    - 死锁
    - 内存管理
- 设计模式【建议学】
    - 单例模式
    - 代理模式
    - 工厂模式
    - 装饰者模式
    - 观察者模式
    - 策略模式
    - 门面模式
- Git 版本控制【必学】
    - 常用命令
- Linux 服务器【建议学】
    - 常用命令
- 正则表达式【建议学】

编程导航(codefather.cn)
编程导航(codefather.cn)

学习建议

这个阶段的重点是深入学习 JavaScript，包括 ES6+ 新特性、异步编程（Promise、async/await）、闭包、原型链等概念。建议看《JavaScript 高级程序设计》这本书，是前端必读经典。

学习资源

- 《JavaScript 高级程序设计》：前端必读经典
- 《JavaScript 忍者秘籍》：深入理解 JavaScript
- ES6 入门教程：阮一峰老师的 ES6 教程
- 算法学习动画教程：鱼皮出品，用动画讲解算法

阶段 3：前端工程化（30-70 天，仅供参考）

前端工程化是使用软件工程的技术和方法来进行前端的开发流程、技术、工具、经验等规范化、标准化，主要目的是提高开发效率，减少不必要的重复工作。这个阶段你将学习主流前端框架（Vue 或 React）、脚手架、组件库、构建工具等，并能开发完整的前端项目。

学习目标

至少学会一门主流的前端开发框架（Vue 或 React），并配合脚手架、组件库、工具等从 0 开始独立搭建并开发一个完整的前端网站。要求遵循企业开发规范，将项目代码提交到代码仓库中，并独立发布上线。

知识点

研发流程【必学】：

1. 技术选型
2. 初始化
3. 开发
4. 本地测试
5. 代码提交
6. 编译、打包、构建
7. 部署
8. 集成测试
9. 发布上线
10. 监控运维

代码托管【必学】：

集中存储、备份你的代码，还能和团队成员协作开发。

- GitHub【必学】
- Gitee【建议学】
- GitLab【建议学】

Node.js【必学】：

一个开源和跨平台的 JavaScript 运行时环境，是前端开发的基础工具。

- 包管理工具【必学】
  - npm【必学】
  - yarn【必学】
  - npx【建议学】
- Node.js 开发框架【可不学】

编程导航(codefather.cn)

    - Express
    - Koa
    - Egg

前端开发框架【必学】：

解决特定的问题，提高开发效率、简化代码复杂度。

- CSS 框架【建议学】
    - BootStrap
    - Tailwind CSS
- JavaScript 框架【必学，至少学一个】
    - Vue【推荐新手】
        - Vue Router【必学】
        - Vuex / Pinia【必学】
    - React【推荐有基础的同学】
        - React DOM【必学】
        - React Router【必学】
        - Redux【建议学】
        - MobX【建议学】
        - React Hooks【必学】
    - Angular【可不学】
        - RxJS
        - NgRx
    - Svelte【可不学】
    - UmiJS【建议学】

前端封装库【建议学】：

- 组件库【必学，至少学一个】
    - Element UI（Vue）【推荐】
    - Vant UI（Vue，移动端）【建议学】
    - Ant Design（React）【推荐】
    - LayUI【可不学】
    - Material UI【可不学】
- 数据可视化库【建议学】
    - Apache ECharts【推荐】
    - AntV【建议学】
    - HighCharts【可不学】
    - D3.js【可不学】
- 常用组件和插件【建议学】
    - 富文本编辑器
    - 弹窗
    - 轮播图
- 工具库【建议学】
    - Axios【必学】
    - Lodash【建议学】
    - 时间处理 Moment.js【建议学】
    - jQuery【可不学】
- 动效库【可不学】

编程导航(codefather.cn)
编程导航(codefather.cn)

    - Animate.css
    - Ant Motion
- 字体图标库【建议学】
    - IconFont【推荐】
    - IconPark【建议学】
    - Font Awesome【建议学】

脚手架【必学】：

快速生成新项目的目录模板，提升开发效率和开发舒适性。

- Vue CLI【必学】
- create-react-app【必学】
- Vite【推荐】
- Yeoman【可不学】

前端架构设计【建议学】：

一系列工具和流程的集合，旨在提升前端代码的质量，并实现高效可持续的工作流。

- 模块化【必学】
- 组件化【必学】
- MVVM【建议学】
- 设计原则【建议学】
- SPA 单页应用【必学】
- 多页应用【建议学】
- 前端路由【必学】
- PWA【可不学】
- 服务端渲染（SSR）【建议学】
    - Next.js（React）
    - Nuxt.js（Vue）
- BFF（Backend For Frontend）【可不学】
    - GraphQL
- 微前端【可不学】
    - qiankun
    - single-spa

CSS in JS【建议学】：

用 JavaScript 来写 CSS。

- CSS 模块化【建议学】
    - CSS Modules【推荐】
    - styled-components【建议学】
    - Styled JSX【建议学】
    - Emotion【可不学】
- 内联样式
- 声明样式
- 引入样式

CSS 预编译【建议学】：

使用新的扩展语言以增强 CSS 的能力，并通过预编译的方式将其转换为浏览器认识的 CSS 代码。

编程导航(codefather.cn)
编程导航(codefather.cn)

- SASS【推荐】
- PostCSS【建议学】
- Stylus【可不学】
- LESS【可不学】

开发调试【必学】：

本地开发时热更新，提升开发效率。

- webpack-dev-server【必学】
- serve【建议学】
- 内网穿透【可不学】
  - Ngrok
  - NATAPP

测试【建议学】：

- 测试分类
  - UT 单元测试【建议学】
  - SIT 系统集成测试【可不学】
  - E2E 端到端测试【建议学】
  - UAT 用户验收测试【可不学】
- Mock【建议学】
  - Mock.js（通过随机数据，模拟各种场景，增加单元测试的真实性）
- 测试框架【建议学】
  - Jest【推荐】
  - Puppeteer（Headless Browser）【建议学】
  - Enzyme【可不学】
  - Mocha【可不学】
  - Jasmine【可不学】

代码质量【必学】：

- 开发规范【必学】
  - CSS Style Guide
  - JavaScript Style Guide
  - Airbnb 代码规范：https://github.com/BingKui/javascript-zh
- 类型校验【建议学】
  - TypeScript【推荐】
- 代码检查【必学】
  - ESLint
  - StyleLint
- 代码风格【必学】
  - Prettier（根据配置自动格式化代码，统一格式）
- 提交规范【建议学】
  - 约定式提交
- 提交检查【建议学】
  - pre-commit
  - husky

构建工具【必学】：

编程导航(codefather.cn)
编程导航(codefather.cn)

- 自动化构建【建议学】
  - npm script【推荐】
  - Gulp【可不学】
  - Grunt【可不学】
- 打包工具【必学】
  - Webpack【推荐】
  - Vite【推荐】
  - Rollup【建议学】
  - Parcel【可不学】
  - Snowpack【可不学】

CI / CD【建议学】：

通过在应用开发阶段引入自动化来频繁向客户交付应用。

- GitLab CI【建议学】
- Jenkins【建议学】

部署【建议学】：

- Web 服务器【建议学】
  - Nginx【推荐】
    - 高性能的 HTTP 和反向代理 web 服务器
    - 反向代理
    - 解决跨域
    - 改写请求
  - Apache【可不学】
- 容器【建议学】
  - Docker【建议学】
    - 容器化部署，将代码及其所有依赖关系打包
  - Kubernetes【可不学】
- 部署策略【建议学】
  - 全量发布
  - 蓝绿部署
  - 滚动发布
  - 灰度发布

监控告警【建议学】：

- 前端埋点
- 错误监控
- 性能监控
- 行为监控

学习建议

1）必须学一个主流框架：Vue 或 React 二选一，建议新手从 Vue 开始学，因为 Vue 的学习曲线更平缓、中文资料更多。

2）做一个完整项目：学完框架后，一定要从 0 到 1 做一个完整的项目。强烈推荐跟着鱼皮的 前端项目教程 学习，涵盖 Vue 和 React 两个方向，从新手入门到企业级项目都有。

编程导航(codefather.cn)
编程导航(codefather.cn)

3）补充相关技术学习：前端工程化涉及很多技术栈，可以结合编程导航的其他学习路线系统学习，比如 TypeScript、Node.js、小程序开发、Next.js 等。

学习资源

框架和组件库：

- Vue 3 官方文档：Vue 官方中文文档，必读
- React 官方文档：React 官方中文文档
- Element UI 官方文档：Vue 组件库
- Ant Design 官方文档：React 组件库
- 黑马程序员 React18 入门到实战：React + Hooks 核心基础到项目实战
- 2025 最新 React 全家桶项目实战：300 小时，从零到全栈

Node.js：

- Node.js 官方教程：官方中文教程
- Node.js 入门教程：社区入门指南
- Node.js 学习路线：系统学习 Node.js

工具和架构：

- 服务端渲染 SSR：SSR 概念和实践
- BFF 架构介绍：Backend For Frontend
- 微前端架构：微前端概念和实践
- CSS in JS：阮一峰的 CSS in JS 教程

部署相关：

- Nginx 学习路线：系统学习 Nginx
- 腾讯云动手实验室：Nginx 实战练习
- Docker 从入门到实践：Docker 容器化
- Docker 容器化学习路线：系统学习 Docker

相关学习路线：

- TypeScript 学习路线：必学，现代前端开发标配
- Next.js 全栈开发学习路线：React 服务端渲染框架
- 小程序开发学习路线：微信小程序开发
- 移动端 H5 开发学习路线：移动端网页开发
- Electron 桌面应用开发学习路线：跨平台桌面应用

阶段 4：项目实战（30-60 天，仅供参考）

做项目是学好前端的关键！学完框架后，要立即通过实战项目巩固所学知识。

学习目标

通过实战项目巩固所学知识，积累项目经验，提升解决实际问题的能力，为后续的优化和求职准备优质的简历项目。

项目推荐

鱼皮原创前端项目（强烈推荐，按难度排序）：

新手入门项目（建议全部完成）：

1. 用户中心项目：React 前后端分离项目，适合前端新手入门，系统学习完整的项目开发流程和上线方法

编程导航(codefather.cn)

2. AI 程序员技术练兵场：Vue 3 + AI 应用开发，实战 LangChain4j AI 框架、结构化输出、提示词工程
3. 闯关式 SQL 自学网：Vue 3 纯前端项目，实战 Monaco Editor、WebAssembly

企业级项目（选择 2 - 3 个）：

1. 智能协同云图库：Vue 3 企业级项目，实战 WebSocket 实时通讯、权限控制、DDD 领域驱动设计
2. 智能面试刷题平台：React + Next.js 服务端渲染项目，实战 Redis 多级缓存、Elasticsearch 搜索
3. AI 答题应用平台：React 跨端小程序开发 + Vue3 网站开发，实战分库分表、SSE 实时推送
4. SQL 数据生成平台：React 智能化数据生成工具，实战设计模式、FreeMarker 模板引擎
5. AI 零代码应用生成平台：微服务全栈项目，实战 AI 智能体、工作流编排、Nginx、响应式编程

前端专项（进阶学习）：

1. 复杂 SQL 生成器：Vue 3 + TypeScript，实战复杂业务逻辑处理
2. 测试鸭：React 交互式学习网站，学习状态管理、模块化设计
3. Web 终端项目：Vue 3 全栈项目，学习组件化设计、TypeScript、插件化架构

免费项目合集（快速入门）：

- 2025 最新 20 个 Web 前端实战项目：练完即可就业
- 2025 最新 15 个 Web 前端实战项目：从入门到进阶
- 2025 最新 10+ Web 前端实战项目：基础到框架

学习建议

1）建议按难度递进：先做新手入门项目（至少 2 个），掌握基本的项目开发流程，再挑战企业级项目。

2）每个项目都要完整完成：不要只看教程不动手，一定要自己敲代码、调试、上线。做完项目后，在阶段 5 可以对项目进行性能优化。

3）做完项目要总结：写项目总结文档，画架构图，准备好面试时的项目介绍。

4）适当优化和扩展：在原有项目基础上增加自己的想法，提升简历区分度。

学习资源

- 鱼皮项目学习建议（必读）：详细的项目学习方法和建议

阶段 5：前端优化（15-20 天，仅供参考）

前端优化是指通过各种分析和优化手段，提高网站的性能和用户体验。这个阶段可以将阶段 4 做的项目进行优化。

学习目标

实践前端优化方法，从多个方面优化自己做过的项目，最好能将优化后的网站和原网站进行对比，得到一些数据和体验上的明显提升。

知识点

性能优化【建议学】：

- 性能指标【必学】
  - FP（First Paint）：从开始加载到浏览器首次绘制像素到屏幕上的时间
  - FCP（First Contentful Paint）：浏览器首次绘制来自 DOM 的内容的时间
  - FMP（First Meaningful Paint）：页面的主要内容绘制到屏幕上的时间
  - FSP（First Screen Paint）：页面从开始加载到首屏内容全部绘制完成的时间
  - TTI（Time to Interactive）：网页第一次完全达到可交互状态的时间点

编程导航(codefather.cn)

- 优化手段【建议学】
  - 性能监控（Performance API）
  - 样式优化
  - JavaScript 优化（防抖、节流）
  - 代码分割
  - 资源压缩
  - 打包优化
  - 服务器优化
  - 缓存优化（Service Worker）
  - 动画性能优化
  - dns-prefetch
  - Lazy loading（懒加载）
  - 优化启动性能（异步化）
  - 渲染优化
  - 网络优化
  - 移动端性能优化
  - CDN（内容分发网络）

用户体验【建议学】：

- 设计系统
- 骨架屏

兼容性【建议学】：

- 浏览器兼容性【建议学】
  - normalize.css
  - html5shiv.js
  - respond.js
  - Babel
  - Polyfill
- 屏幕分辨率兼容性【建议学】
- 跨平台兼容性【建议学】

SEO【建议学】：

搜索引擎优化，利用搜索引擎的规则提高网站在有关搜索引擎内的自然排名。

- robots.txt
- description
- keywords
- SiteMap
- HTML 标签优化
- 站内链接建设
- 友情链接

安全【建议学】：

- XSS（跨站脚本攻击）【必学】
- CSRF（跨站请求伪造）【建议学】
- 反爬虫【可不学】
- SQL 注入【建议学】
- DDoS【可不学】

编程导航(codefather.cn)
编程导航(codefather.cn)

学习建议

1）性能优化是面试重点：前端性能优化是大厂面试必问的内容，一定要系统学习。

2）从实际项目入手：学习性能优化最好的方法就是优化自己做过的项目。使用 Chrome 的 Lighthouse 工具分析项目的性能。

学习资源

- BootCDN：免费 CDN 加速服务
- 百度搜索资源平台：SEO 优化指南

阶段 6：前端生态（选学，20-30 天）

前端领域的范围越来越广，出现了更多前端工程师需要了解和关注的技术，包括静态站点构建、移动应用开发、桌面应用、小程序、Serverless、云开发、低代码等。

学习目标

能用学到的前端技术去做更多的事情，比如开发个人博客、文档网站、小程序、桌面应用等。

知识点

静态站点构建【建议学】：

- 开发框架【可不学】
  - React Static
- 博客【建议学】
  - Gatsby.js
  - Docusaurus
  - Hugo
  - Hexo
- 文档【建议学】
  - Docsify【推荐】
  - VuePress
  - Dumi
  - JekyII

大前端【建议学】：

前端技术的融合更加紧密，应用的领域也更加广泛，除了网站外，还可以用前端技术跨平台开发 Android、iOS、小程序、虚拟现实、增强现实等。

- 移动应用【建议学】
  - Hybrid
  - WebView
  - React Native
  - Flutter
- 移动应用打包【可不学】
  - Weex
  - Cordova
  - Phonegap
  - Ionic

编程导航(codefather.cn)

- 桌面应用【建议学】
  - Electron【推荐】
  - NW.js
  - Proton Native
- 小程序【建议学】
  - 原生小程序【建议学】
  - WebView
- 跨端开发框架【建议学】
  - uni-app【推荐】
  - Taro【推荐】
  - Flutter
  - Chameleon
  - Wepy
  - Rax
- 移动端调试【建议学】
  - Chrome Dev Tools
  - Android Simulator
  - IOS Simulator

Serverless【可不学】：

一种构建和管理基于微服务架构的完整流程，允许你在服务部署级别而不是服务器部署级别来管理你的应用部署。

云开发【可不学】：

云端一体化的后端云服务，采用 Serverless 架构，免去了移动应用构建中繁琐的服务器搭建和运维。

- 腾讯云云开发
- 阿里云云开发

低代码【可不学】：

无需编码（零代码）或通过少量代码就可以快速生成应用程序的开发平台。

- 腾讯云低码
- 阿里宜搭

学习建议

1）按需学习即可：前端生态的内容非常广（小程序、App、桌面应用、Serverless 等），不需要全学，根据你的兴趣和求职方向选择性学习。

2）小程序开发是很多公司的需求，建议学习一下。

学习资源

相关学习路线：

- 小程序开发学习路线：微信小程序、支付宝小程序等
- React Native 学习路线：跨平台移动应用开发
- Flutter 跨平台开发学习路线：Google 跨平台框架
- Electron 桌面应用开发学习路线：跨平台桌面应用
- 鸿蒙应用开发学习路线：华为鸿蒙系统应用开发

编程导航(codefather.cn)

阶段 7：前端求职

求职是一个漫长的过程，建议拉长战线，尽早准备。这个阶段主要是准备简历、刷面试题、参加面试，最终找到理想的工作。

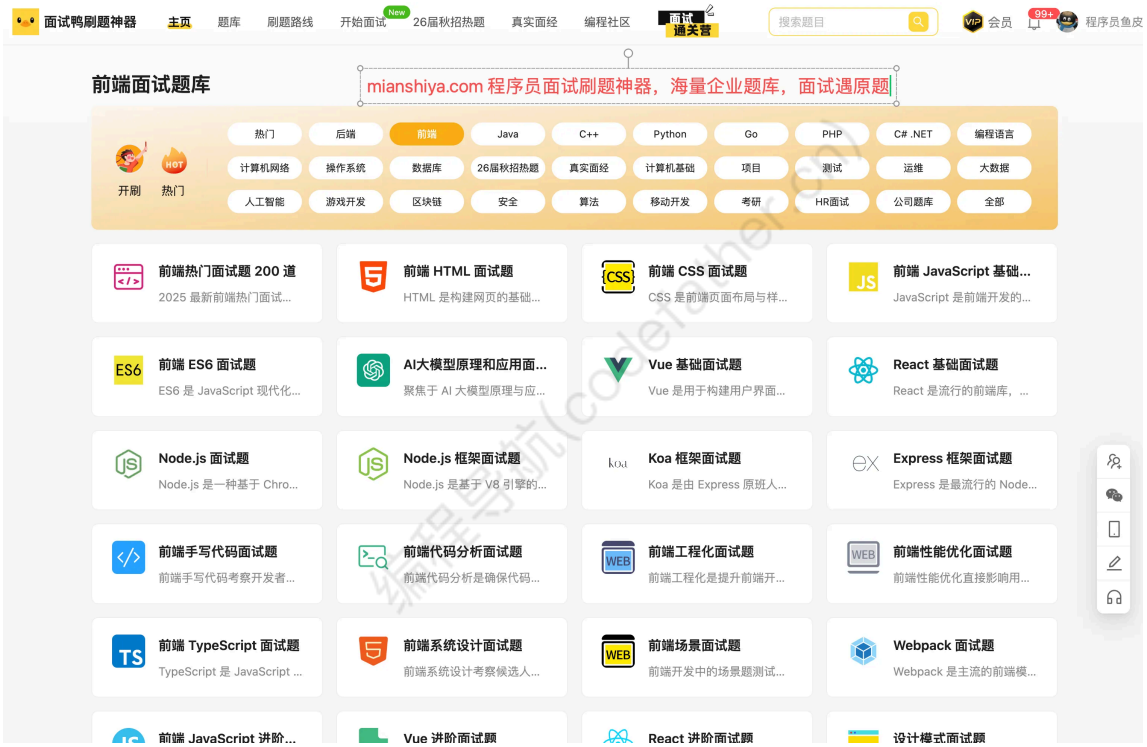
学习建议

1）提前做规划：尽早做规划，明确自己的求职目标（Web 前端、小程序、全栈等），针对性地准备。建议至少提前 3 个月开始准备。推荐阅读鱼皮的 保姆级求职指南。

2）打磨简历：简历是你的第一张名片。前端是个看作品的行业，简历上一定要有 2-3 个拿得出手的项目，最好能部署上线，在简历上提供访问链接。建议使用 老鱼简历 制作简历，关于如何写好简历，推荐学习鱼皮的 保姆级写简历指南。



3）多刷面试题：使用 面试鸭 刷题，支持按公司、题型、难度筛选。前端的面试题主要考察 HTML、CSS、JavaScript 基础，框架（Vue/React），以及计算机基础、算法、项目经验等。



4）多看真实面经：真实面经大全、几百场真实面试视频。

5）善用 AI 模拟面试：使用 AI 模拟面试 提前练习。



6）看面试题讲解视频：关注 面试鸭 B 站账号，有大量面试题讲解视频。

更多求职干货：编程导航求职干货分享

学习资源

前端基础面试题：

- 前端 HTML 面试题 - 面试鸭
- 前端 CSS 面试题 - 面试鸭
- 前端 JavaScript 基础面试题 - 面试鸭
- 前端 JavaScript 进阶面试题 - 面试鸭
- 前端 ES6 面试题 - 面试鸭
- 前端 TypeScript 面试题 - 面试鸭

前端框架面试题：

- Vue 基础面试题 - 面试鸭
- Vue 进阶面试题 - 面试鸭
- Vue3 面试题 - 面试鸭
- React 基础面试题 - 面试鸭
- React 进阶面试题 - 面试鸭

综合面试题：

- 前端经典面试题合集 - 面试鸭
- 前端代码分析面试题 - 面试鸭
- 前端手写代码面试题 - 面试鸭
- 前端场景面试题 - 面试鸭
- 前端系统设计面试题 - 面试鸭
- 前端工程化面试题 - 面试鸭
- 前端性能优化面试题 - 面试鸭

大厂面经：

- 26 届秋招前端热题 - 面试鸭
- 前端热门面试题 200 道 - 面试鸭

阶段 8：持续学习

前端未来可能的发展趋势和热门技术，建议关注但不用急着深入学习。

学习目标

关注前沿技术，持续进步。

知识点

前端未来趋势【可不学】：

- WebAssembly：高性能 Web 应用
- 智能 UI：AI 辅助 UI 设计和开发
- React Server Component：React 服务端组件
- CSS Houdini：CSS 引擎 API
- WebComponents
  - HTML templates（HTML 模板）

编程导航(codefather.cn)

- Custom elements（自定义元素）
- Shadow DOM（影子 DOM）

学习建议

保持对前端新技术的关注，但不要盲目追新，先打好基础，再根据实际需求学习新技术。

鱼皮的前端学习经历

虽然上面整理的学习路线知识点非常多，但是大家也别太担心，其实按照一定顺序和方法来学习，并不会很难，聊聊我的学习经历。

1、快速入门，培养兴趣

刚开始学习前端时，一定要以培养兴趣、快速入门为目标。我入门前端时，由于没有基础、啥都不懂，因此选择在线实战教程网站来学习，跟着教程案例一行行把代码敲下来，实时浏览显示效果，很轻松地就把 HTML、CSS、JavaScript 的语法基础过了一遍。但是这时，我对看过的知识点没有什么印象，基本是看了就忘，当我想要自己写一个网站，也根本无从下手。

于是，我开始试着死记硬背代码，结果发现背了之后还是不会写。那干脆一不做二不休，直接打开编辑器和教程网站，把之前在线实战时敲过的代码复制粘贴到编辑器中，然后再修改保存，到浏览器中打开文件。这样虽然要在编辑器和浏览器中来回切换，但起码能看到自己的网页文件运行的效果了。后来，我利用浏览器开发者工具提升了自己开发网站的调试效率，敲出的代码达不到满意的效果时，就再去查，再改，再浏览，最后终于完成了一个留言板网站！

通过这件事，我明白了一个道理，既然记不住知识，那就先别强行记忆，上手多写，忘了就去查不就好了？不要担心自己知识不够做不出来，只要你去试，都一定能实现。

2、多看多写，巩固基础

在这种操作下，我虽然能使用基础三件套开发简易网站了，但做的很慢，还老出问题。于是，我开始完整地看了几本前端入门和项目实战书籍，有三件套入门的，也有响应式网站设计的，还有 JavaScript 综合讲解的，以及一套完整的视频教程，通过这些来复习巩固之前学过的知识，打好了基础，并且了解了更多实战教程上没见到的知识。

看书和视频的过程中，我跟着做了几个简易的网站，也是边看边做。在这之后，我发现自己渐渐地能够脱离文档来写网站了！

3、框架学习，学以致用

但是，总感觉开发网站时，什么都要自己写，重复的代码写一大堆，很麻烦。于是，我试着上网搜了解决这些问题的方法，发现可以用组件库减少重复编写 HTML 和 CSS 代码，于是学了 LayUI，其实和之前一样的，就是打开官网找到要用的组件，然后复制到项目中修改就行了，写的多了竟然还真记住了一些（虽然这个没必要记）。

界面和样式代码简化了，但是 JavaScript 写的我依旧头疼。于是我决定进入下一个阶段的学习，当时先看了本书叫《锋利的 jQuery》来学习 jQuery，学会之后使用它趁热打铁做了几个网站，发现的确能大大简化繁琐的 JavaScript DOM 操作，让我尝到了甜头，又有了继续学习前端的动力。

在那之后大概半年，我开始学习主流前端框架 Vue，也是跟着视频教程学，并且结合 ElementUI 组件库开发了一个响应式网站。不得不说，习惯了用原生 JavaScript 和 jQuery 开发后，再去用 Vue 开发，真的是爽飞了，小有成就感。

4、根据需要，自主学习

在那之后，我就是通过不断地做项目来积累经验，既然编程知识太多学不完，那我就随需随学，逐渐拓宽自己的知识面。举个例子，我要做一个移动端 H5 页面，那就去搜 Vue 的移动端组件库，学会了 MintUI、VantUI 等；项目中要处理日期，那我就去搜日期处理库，学会了 Moment.js；项目重复文件和代码太多，我就去网上搜一些组件化、模块化的最佳实践；项目代码太大、加载太慢，我就去网上搜一些前端压缩、性能优化的方式；要把开发好的项目变成

APP，那我就去网上搜到了 Cordova 等打包工具；跟其他同学一起做项目时比较混乱、开发效率低，我就去搜了一些代码规范、前端工程化等；遇到 Bug 时，也是自己去网上搜索解决方案，并且还要看看有没有知识点是我遗漏的，我会再去弥补学习。通过不断实践、不断遇到问题和自主解决问题，我始终保持着学习的积极性，学到了越来越多前端的知识，并通过归纳总结来加深印象。

5、渐入佳境，拓宽视野

在那之后，我学前端技术非常快了，像 React、TypeScript、Redux、Dva、Umi、Lerna 等等都随用随学，随学随用。此外，我不仅仅关注前端网页开发本身，还同时关注前端的生态和发展，学习了微前端、Serverless、云开发、低代码等技术，因此个人的能力提升也很大。如今在鹅厂工作中也是，面对各种不同技术栈的前端项目，我都能迎难而上，毕竟不会就学嘛！

哦，写得渐入佳境，我都忘了，自己是一名后端工程师（笑）。所以本路线是结合了我的前端学习经历和大量资料整理而成的，希望能对大家有所帮助。

6、学习路线总结

总结一下，学前端的路线其实就一句话：**基础三件套 => 巩固基础 => 学习框架 => 多做项目 => 按需学习 => 拓宽视野**。

还有一个学任何编程技术都要注意的点：**全程多敲代码！多敲代码！多敲代码！**

持续学习资源

前端职业发展

1. 初级前端工程师：能通过 Vue/React 框架完成公司的需求，比如还原设计稿、开发交互、前后端联调等。重点是要熟练使用框架和组件库，能独立完成功能开发。
2. 中级前端工程师：除了能完成需求，还要能搭建项目架构、开发通用组件、进行性能优化、解决疑难问题等。需要熟练使用前端工程化工具（Webpack、Vite 等），了解框架原理。
3. 高级前端工程师 / 架构师：要了解常用框架的运行原理和机制，能做技术选型、架构设计。最好有一个自己擅长的领域（如低代码、可视化、性能优化等），并且积极参加开源项目和技术分享。

优秀前端工程师的特质

- 深厚的技术功底
- 良好的编码习惯和代码规范
- 善于发现和解决问题
- 思维灵活，追求用户体验
- 紧跟前端技术前沿
- 有创造力和行动力
- 有全栈意识
- 懂得团队协作和沟通
- 责任心强，有全局观

官方资源

- MDN Web 开发文档：前端官方文档，必读
- Node.js 官方文档：Node.js 中文文档
- Vue 官方文档：Vue 中文文档
- React 官方文档：React 中文文档

学习平台

编程导航(codefather.cn)

- 编程导航：学习路线、项目教程、面试题一站式平台
- freeCodeCamp 在线编程：在线实战学习
- 阿里云前端实战：在线动手实验
- 腾讯云动手实验室：云端实验环境

技术博客和社区

- 腾讯 Alloy Team：腾讯 Web 前端团队
- 百度 FEX：百度 Web 前端研发部
- 京东凹凸实验室：京东前端团队
- 有赞技术团队：有赞前端

前端工具和资源

- Best of JS：JavaScript 明星项目
- Vue.js Examples：Vue 示例
- Codewars：JavaScript 练习

写在最后

前端开发是一个充满创造力和成就感的领域，能让你的想法快速变成现实。虽然前端的知识点很多、技术更新很快，但只要按照正确的路线学习，一步步积累，一定能成为优秀的前端工程师。

学习要点总结：

- 前期学习以培养兴趣为主，不要过于追求深层理解
- 前端学习不能靠死记硬背，要多敲代码、多做项目实践
- 不要急于求成，踏实积累才是硬道理
- 善用 AI 工具，提升学习和开发效率

特别是在 AI 时代，前端开发有了更多可能性。AI 辅助编程能大大提升开发效率，AI 应用需要优秀的前端界面。掌握前端 + AI，将会拥有更强的竞争力。

希望这份学习路线能够帮助大家少走弯路，更高效地成为前端工程师。加油小伙伴们！💪



编程导航(codefather.cn)

程序员必备资源

1）程序员学习交流圈：极客教程、实战项目、求职宝典

2）程序员面试八股文：实习/校招/社招高频考点、企业真题解析

3）程序员写简历神器：专业模板、丰富例句、直通面试

4）AI 知识资源大全：前沿技术、最新 AI 资讯、提示词大全

5）1 对 1 模拟面试：实习/校招/社招面试拿 Offer 必备

编程导航(codefather.cn)

编程导航(codefather.cn)